TRAIN YOUR AI · COMPANION GUIDE

# Give your AI a **home**.

Take on any complicated project — and the scattered pile of documents behind it — with a workspace that reads, evaluates, and writes with you, from the first question to the finished product.

Train Your AI · follow along on whatever AI tool you already have open.

---

**IN A HURRY? DO THIS TONIGHT**
Open ChatGPT, Claude, or Gemini → make a **Project** (Claude / ChatGPT) or a **Gem** (Gemini) → paste in the instruction file from Move 2 → upload the documents you're working from → then start every chat from inside it. That's the whole move — it's Rung 2, the right home for almost everyone. Everything below is *why* it works and how to do it well.

01 · THE THREE TAXES

## Most of what you need is already on your computer. You're just paying to move it around.

Reports. Journal articles. Slide decks. Policy memos. Notes from a dozen meetings. It piles up faster than you can read it, and most of it feels like noise you're trying to keep up with.

It isn't noise. It's the raw material for whatever complicated project is in front of you — **a program review, a policy change, a decision brief, a literature review, a paper.** Much of what you'll build on is already sitting in a folder somewhere on your computer — your material, your notes, your standards; the sources for your topic you'll still go and find. Either way, the hard part was never the material. It's what it *costs* you to *use* it — and that cost comes as three quiet taxes you pay all week without noticing:

| | |
|---|---|
| RELOAD | **You re-explain yourself every time.** Every fresh chat, you pour the same funnel back in — the format, the style, the example of good, the brain-dump of what you know. The AI forgot all of it the moment the last chat closed. |
| FILE | **You re-find your own work.** Where did I put that reading? Which draft is the latest? `OER_v3_FINAL_FINAL.docx`. |
| RELAY | **You re-say it for each new reader.** The same point, retranslated for a peer, a boss, a patient — reformatted into a memo, a slide, an email, every time from scratch. |

Most people learn to use AI one careful prompt at a time — into an AI that forgets you the moment the chat closes. That works, but it pays all three taxes in full. This guide is about the next move: **give the AI a persistent home** — a place that already holds your material, your topic, and your standards — so you stop starting cold.

The whole method is three moves. You'll do them in order, and the rest of this guide is those three moves spelled out with the exact prompts to paste:

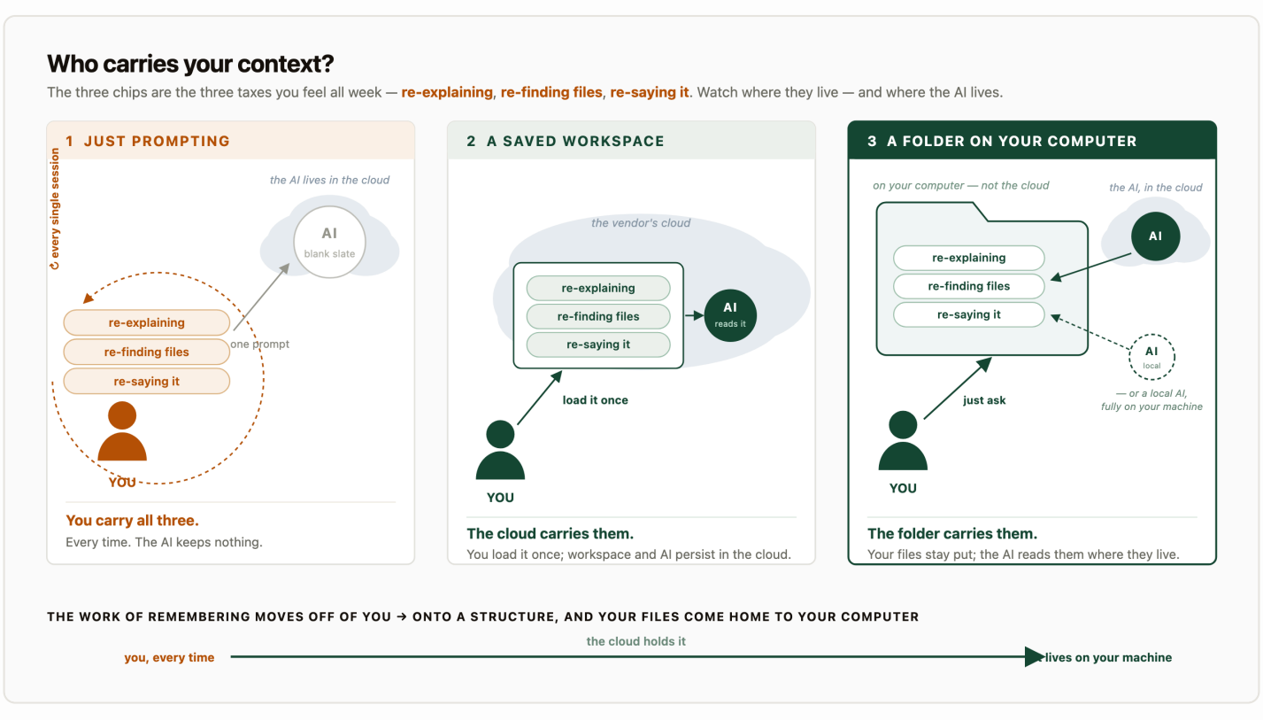
| | |
|---|---|
| DISCOVER | **You don't have to know how to organize it first.** Point the AI at the pile and let it help you see the shape — what's in there, what the project really turns on, what supports it. |
| SCAFFOLD | **Once you can see the shape, build the folder.** A place for your sources, a place for your ideas, a place for the rules. Every file gets one home. |
| ENFORCE | **Then force everything through the standard and iterate.** Read it hard, draft grounded, review until it holds — until you'd defend every line. |

First, though, you have to decide *where* the AI's home will be — because that decides which of the three taxes you actually stop paying.

02 · THE SHIFT

## There are three ways to give the AI a home. The difference is who does the remembering.

The three taxes — reloading, re-finding, re-saying — don't disappear. In every setup below they still exist. The only thing that changes is *who carries them*: you, a box in the cloud, or a folder on your own computer.


Who carries your context?
The three chips are the three taxes you feel all week — re-explaining, re-finding files, re-saying it. Watch where they live — and where the AI lives.
1 JUST PROMPTING
every single session
the AI lives in the cloud
AI
blank slate
re-explaining
re-finding files
re-saying it
one prompt
YOU
You carry all three.
Every time. The AI keeps nothing.
2 A SAVED WORKSPACE
the vendor's cloud
re-explaining
re-finding files
re-saying it
AI
reads it
load it once
YOU
The cloud carries them.
You load it once; workspace and AI persist in the cloud.
3 A FOLDER ON YOUR COMPUTER
on your computer — not the cloud
the AI, in the cloud
AI
re-explaining
re-finding files
re-saying it
AI
local
— or a local AI, fully on your machine
just ask
YOU
The folder carries them.
Your files stay put; the AI reads them where they live.
THE WORK OF REMEMBERING MOVES OFF OF YOU → ONTO A STRUCTURE, AND YOUR FILES COME HOME TO YOUR COMPUTER
you, every time
the cloud holds it
lives on your machine

*Left to right, those three panels are your three rungs. Same three taxes in each — but the work of remembering slides off of you and onto a structure you own. That slide, from left to right, is the whole idea of this guide.*

Picture these three setups as a ladder: *just prompting* at the bottom, a *saved workspace* in the cloud in the middle, and a *persistent home on your own computer* at the top. You do not have to climb to the top. Higher rungs are more capable and keep your work more firmly under your control — but they ask more of you to set up. Most of you should stand on the middle rung. Here's each one in full:

### RUNG 1 Upload into a chat

Drag your documents into a normal chat and ask questions. Fine for a quick, one-time task — but it isn't really a *home*. When the chat closes, the context is gone; next time you re-upload and re-explain from scratch. Nothing is organized, and nothing persists.

| CODING? | WHERE YOUR WORK LIVES | PERSISTS? |
|---|---|---|
| None | The chat window | No — resets each chat |

GOOD FOR
A one-off question

### RUNG 2 A project workspace START HERE

A saved workspace where you write the AI's instructions once and upload your knowledge once — and every new conversation starts *warm*, already knowing your project and your material. No coding. This is the right home for almost everyone.

| CODING? | WHERE YOUR WORK LIVES | PERSISTS? |
|---|---|---|
| None | The vendor's cloud | Yes — across every chat |

GOOD FOR
Real writing, and beyond

**What it's called: Gemini Gems** (free) · ChatGPT **Projects** (works on the free tier) · Claude **Projects** (free, capped at a handful) · Microsoft **Copilot** agents (needs a paid work license). On the government side, **GenAI.mil** now has a no-code **Agent Designer** that follows this same save-your-context pattern.

### RUNG 3 A workspace on your own computer

Rungs 1 and 2 hand you text to copy out. Rung 3 does more than generate — it *acts*. Because it runs on your own computer, it can do anything your command line can: update a document in place instead of making a new copy every time, run a program, render a polished PDF, reorganize a folder, even send an email on your behalf. Everything stays on your hard drive; nothing lives in someone else's window. The most powerful and most private option, with the steepest ramp.

| CODING? | WHERE YOUR WORK LIVES | PERSISTS? |
|---|---|---|
| Some comfort with a terminal helps | Your own hard drive | Yes — it's just your files |

GOOD FOR
Power users; sensitive work

**Where to start:** if you want to try this rung, **Cline** is the friendliest on-ramp for a non-coder. (Alternatives, roughly hardest-working to simplest: Claude **Code**, Gemini **CLI**, OpenAI **Codex CLI**, **Cursor**; and Claude **Cowork** / Claude **for Chrome**, both still in preview. The exact tools churn — the discipline doesn't.) For data that legally can't leave your computer, a **local model** run with **Ollama** or **LM Studio** keeps everything — files *and* processing — on your machine.

**THE MENTAL MODEL — ABOVE AND BELOW THE WATERLINE**
The word "prompt" only names the tip of the iceberg — the sentence you typed. Below the waterline sits everything the model actually reads to answer you: the standing instructions, your uploaded files, the earlier turns of the chat, whatever it retrieved. A home is just a way to *own* what's below the waterline instead of retyping it. The rule of thumb: **Always Be Contexting** — never make the model guess at something you could have handed it.

**THE POINT**
The same three moves — discover, scaffold, enforce — work on *every* rung. Pick the lowest rung you're comfortable with today. You can always climb later, and nothing you learn is wasted when you do.

**BEFORE YOU UPLOAD ANYTHING — A HEALTH-CARE WARNING**
"Runs on my computer" is not the same as "stays on my computer." On Rungs 1, 2, and most of Rung 3, your files may sit on your disk but their *contents* are still sent to a cloud model on every request. That means **no PHI, no PII, no patient data, no CUI** goes into any of these — the same rule that applies to any system you haven't been cleared to use. As long as you're working with public or non-sensitive material, you're fine here. The only setup that keeps data truly on the machine is a *local* model (Ollama or LM Studio) — that's the path for genuinely sensitive work.

03 · MOVE 1 · DISCOVER

## Point the AI at the pile and let it show you what's there.

The first move surprises people. You don't sort the pile first and then ask the AI. You gather it all in one place, hand the AI the *un*sorted heap, and let it help you see what's there — including what your paper should be about.

### First, gather the corpus into one folder

Before the AI can help you make sense of your material, it needs to be able to see it. Collect all of it — you'll sort it later:

- **Your standard.** Whatever defines “good” for what you're writing — a style guide, a rubric, a template, a boss's past edits, an example you're matching. Drop it in so the AI writes to it, not to its guess of it.
- **The sources** you'll draw on — papers, reports, policy, prior work in your field.
- **Your own work** — notes, half-formed drafts, meeting minutes. Your half-formed ideas are in there.

Do this yourself — don't wait for someone to hand you a tidy bundle. Whatever system holds your material — a shared drive, an inbox, a wiki, a course site — knowing how to pull the whole thing down is a skill you'll reuse constantly. Most systems let you select everything and download it as one zip; for anything that won't export — a web page, an email, a PDF-only report — open it and use your browser's `Print → Save as PDF`. Drop it all in one place for now — even your `Downloads` folder is fine.

### Then let the AI find your topic

Load your material into your Rung 2 workspace (or point your Rung 3 agent at the folder), then paste this:

PASTE THIS · DISCOVERY

```
Here is everything I've collected on [MY FIELD / PROJECT].
Read my standard or template FIRST — especially what a strong
piece is expected to contain.

Then help me find my angle:

1. Summarize the main themes actually present in this material,
   based on the documents — not general knowledge.

2. Propose 6–8 specific topics or angles that are:
   - relevant to a real, current problem I'd actually address,
   - clearly tied to THIS material (name which document each
     one draws on),
   - narrow enough to cover well in a short piece.

3. For each, note what I could use for background and what a
   recommended course of action might look like.

Before you finalize the list, ask me 2–3 questions about my job,
my interests, and what I actually care about — so the topics fit
me, not a generic reader.
```

**DON'T DUMP EVERYTHING AT ONCE**
Free workspaces cap how many files you can add, and a hundred slide decks bury the few that matter. Load your **standard and key references first**, then the **three to five sources you'll actually cite** — add more only as you use them. Quality of context beats quantity, and it keeps you under the free-tier limits.

Answer its questions honestly. What comes back is a shortlist of topics that are *already* grounded in your material and matched to you. You didn't need to know your topic before you started — the AI plus the corpus surfaced it. Pick one and move on.

**KEEP THE WHEEL**
The AI proposes; *you* decide. Don't take the first topic because it's easy — take the one you can say something real about. Writing is thinking, and it's a lot easier to think about something you care about.

04 · MOVE 2 · SCAFFOLD

## You don't build the folder from scratch — you start from a kit.

The rule is simple: **every file has one home.** When you know where each thing goes, so does the AI. So we hand you a starter workspace — you fill it in rather than architect it.

**DOWNLOAD THE STARTER KIT**
Download the **starter workspace** and unzip it. It already holds the instruction file, a `sources/` folder, the reading checklist, and a blank template for your standard. On Rung 2, upload its files into your Project or Gem; on Rung 3, just open the folder with your agent.

Here's what's inside — four homes, nothing spare:

```
your-workspace/
├── INSTRUCTIONS.md   # the contract — who the AI is, and its hard rules
├── sources/          # the papers & documents you read and cite
├── rubric/           # reading-a-paper.md + your writing standard
└── drafts/           # your writing, kept in versions
```

This is where your Downloads pile gets sorted: the readings you decide to cite move into `sources/`, your standard goes in `rubric/`, and everything else you can leave behind. `sources/` is the folder that fills up — the rest barely changes.

### The instruction file — your AI's contract

This one file changes everything: who the AI is working for, how it must behave, and the lines it can't cross. Open `INSTRUCTIONS.md` (or paste it into your Project/Gem's custom-instructions box) and fill in the one blank:

INSTRUCTIONS.MD · THE CONTRACT

```
WHO YOU ARE
A skeptical, fair editor and reading coach — not a cheerleader.
Your job is to help me understand what I read and defend what I
write. Don't flatter, don't hedge to be polite, don't soften a
real problem, and don't invent one.

WHAT WE'RE DOING
Helping me read and evaluate sources on [TOPIC] and write a
clear, defensible paper, brief, or memo.

HARD RULES — never break these
1. Never invent a statistic, source, quote, or citation. If you
   don't have it, say so and ask me.
2. Trace every factual claim to a real file in /sources. Name it.
3. Don't lead the witness. When you evaluate anything, find what's
   strong AND what's weak. Never just tell me what I want to hear.
4. Flag uncertainty as uncertainty.
5. Track which AI you are and how I used you, so I can disclose it.

VOICE
Plain English. BLUF — bottom line up front. Short sentences,
active verbs. No "utilize," no "leverage," no hedging. If I'd be
embarrassed to say it out loud in a briefing, rewrite it.
```

**ONE FILE, SEVERAL NAMES**
We call this file `INSTRUCTIONS.md` because the name says what it is. It's the same idea as the `CLAUDE.md`, `AGENTS.md`, and `GEMINI.md` files you may have seen — those aren't different things, they're just the specific names certain tools look for and load *automatically*. Same contract, different filename. On Rung 2 you paste it into a custom-instructions box; on Rung 3 you save it under the name your tool expects. More on that in Collaboration & version control.

**THE LINE YOU DON'T CROSS**
Rule 1 — "never invent a source" — is not boilerplate. It's the whole game. You are the author of record. A made-up citation the AI hands you is *your* signature on a lie.

05 · MOVE 3 · ENFORCE

## Force everything through the standard — read, draft, review.

The home is built. Now you keep the standard by running everything through it. Three passes, in order: understand your sources, draft grounded in them, then hand the draft to a skeptic. Each pass has one prompt.

### Read — understand it before you cite it

This is the step people skip — and it's the one that separates real writing from a stack of half-understood quotes. Your workspace turns the AI into a *reading coach*, not a summary machine. Drop a paper into `sources/` and work through it together.

PASTE THIS · READ A PAPER

```
Walk me through the paper in /sources using
rubric/reading-a-paper.md. Don't lead the witness — tell me
what's strong AND what's weak.

Go in this order:
1. In plain English: what does this paper claim, and what
   evidence is it actually resting on?
2. Itemize its assumptions — the ones it states AND the ones it
   doesn't. Which one, if wrong, breaks the conclusion?
3. How strong is it, really — "settles it" or "suggestive"? Why?
4. Give me the exact citation, pulled FROM the paper. Do not
   invent one.

Then quiz me: ask me to state the finding in my own words, and
tell me where I've misunderstood it.
```

That last move — having it *quiz you* — is how you find out whether you understood the paper or just nodded along. If you can't restate the finding with the paper closed, you're not ready to cite it.

### Draft — grounded, not from thin air

With the corpus in the workspace and the instruction file in place, the AI drafts from *your* materials and *your* standard — not from generic internet filler.

PASTE THIS · FIRST DRAFT

```
Using my instruction file (INSTRUCTIONS.md) and the papers in
/sources, draft my [paper / brief / memo] on [TOPIC].

Follow my standard in /rubric. Lead with the main idea (BLUF),
give the background a reader needs, then the detail, then the
recommendation.

Rules:
- Clear, direct style. BLUF. Active voice. Plain words.
- Do NOT invent statistics, sources, or quotes. If you need
  something you don't have, stop and ask me.
- Wherever you use a fact, mark it in brackets with the file it
  came from, e.g. [sources/ayers-2023.pdf], so I can check every
  claim later.

Draft one section at a time and pause after each so I can steer.
```

Work through it section by section. Push back. Cut what's generic. Add what only *you* know from your own experience — that's the part no AI can supply, and it's usually what makes a piece of writing worth reading. You're the writer; the AI is fast hands. Those `[filename]` markers become your citation trail for the next pass.

### Review — you can't referee your own work

The draft is the easy part. What separates real writing from an AI-flavored draft is this loop: hand it to a skeptic, take the hits, and revise. The standard is already in your folder — `rubric/` holds whatever “good” means for your piece — so the AI checks against a real standard, not its guess of one. Open a **fresh** conversation (or a different AI tool) so nothing is invested in defending the draft, and set it loose:

PASTE THIS · ADVERSARIAL REVIEW

```
You are Referee 2 — a skeptical, fair reviewer who wants this
piece to be right, not to feel good. Here is my draft.
Check it against the standard in /rubric.

Mark each item in that standard PASS / WEAK / FAIL and say exactly why.

Then:
1. Flag EVERY factual claim that isn't clearly traceable to one
   of my source files. Treat each as a possible fabrication until
   I prove otherwise.
2. Name the single weakest argument and show how a hostile reader
   would attack it.
3. List what I'd have to defend if my commander asked, "Where did
   each of these numbers and claims come from?"
4. Check the writing against a plain-writing standard — mark
   passive voice, hedging, and needless words to cut.

Do NOT rewrite it for me. Tell me what to fix and let me fix it.
```

Now revise against the critique. Re-run the review. Repeat until it passes clean. Two or three loops is normal — and each one is you doing the thinking, not outsourcing it.

**TWO THINGS WORTH DOING EVERY TIME**
**Disclose the AI.** Good practice — and increasingly required — is to state which AI model(s) you used and how. Keep the note from your instruction file and put it in.

**Own every word.** The output must be edited and represent your final judgment. If you couldn't defend a sentence without the AI in the room, cut it or make it yours.

### Level up: hand it a book you've actually read

A generic "skeptical reviewer" is good. You make it far sharper by giving it a *specific standard* to judge against — a book whose method you've genuinely internalized. Drop the book (or its key chapters) into the workspace and have the AI review your draft against it, one chapter at a time.

**WHY THIS ISN'T CHEATING**
I've been through the complexity of these books — read them, been tested on them, written to their standard. Putting one in the folder *extends my reach; it doesn't replace my judgment.* The book supplies the questions a careful expert reaches only after climbing it — not the five I already ask off the top of my head.

**How.** Turn the book into a checklist: distill each chapter into the principles you want enforced, then run your draft against them, chapter by chapter. It can take all night. So what — that's a thoroughness no tired human sustains at 11pm. The keystone question to insist on: *"Itemize the explicit* ***and implicit*** *assumptions in this draft."* The implicit ones are where the bodies are buried.

**THE ONE RULE THAT KEEPS YOU HONEST**
Never tip the model toward the answer you want. Ask it to find what's *wrong* **and** what's *right* — a neutral interrogation, not a friendly one. Steering it to your prior isn't just weak; it's fooling yourself. As Feynman put it: *"The first principle is that you must not fool yourself — and you are the easiest person to fool."* You're not asking the AI to bless your conclusion; you're building a machine that argues with you.

The books I load as reviewers, in order:

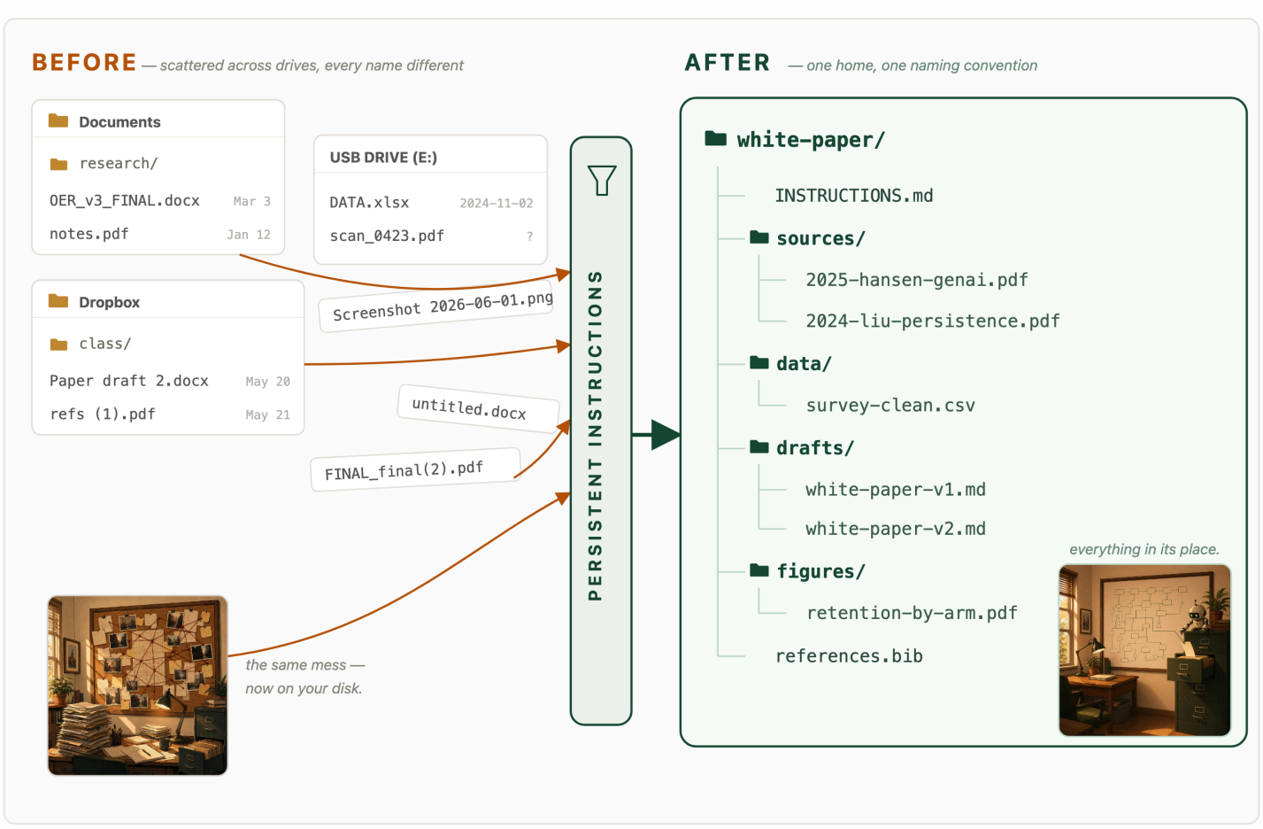
- Bueno de Mesquita & Fowler, *Thinking Clearly with Data* — **start here.** The common language for any quantitative claim: what kind of claim is it, does the evidence support it, could something else explain it, are we measuring what we think we are.
- Scott Cunningham — the "Referee 2" protocol — a worked method for turning an AI into a hostile-but-fair empirical reviewer.
- Huntington-Klein, *The Effect* (free online) — the methods bench for anything claiming cause and effect.

One discipline holds it together: **reason only from sources solid enough to defend.** If your draft leans on a method the book doesn't cover, the AI should say so — flag it as "needs a real source" rather than bluff from a half-remembered fact. That's the same rule as Move 2's "never invent a source," pointed back at your own reasoning.

06 · COLLABORATION & VERSION CONTROL

## Put the home where the team can reach it — and give the AI a filename it already knows.

A workspace on one laptop helps one person. The same workspace on a shared drive, with a standard instruction filename, becomes something a whole team — and a future you — can pick up cold. Two small decisions do it.



*Everything scattered on the left passes through one wall — your persistent instructions — and comes out as one system on the right. Same material, one home, one naming convention.*

### Where the folder lives — collaboration platforms

Keep the workspace on a drive that *everyone works from*, not on one person's desktop. **Box** and **Dropbox** keep one current copy that syncs to the whole team and quietly saves old versions, so there's no "final_v7_JB_edits" drift and you can roll back a bad change. **GitHub** does the same with a full, deliberate history — every change tracked, every version recoverable, built for people working the same files at once. Whichever you use, the win is the same: one home, one current copy, nothing lost.

**VERSION CONTROL, IN ONE SENTENCE**
"Version control" just means the drive remembers every earlier state of your files so you can always go back. You already rely on it every time Box shows "previous versions" — naming it doesn't make it harder.

### What the AI reads first — the persistent instruction file

Your `INSTRUCTIONS.md` works on any rung — on Rung 2 you paste it into a custom-instructions box. But the on-your-computer tools do something better: give the file a conventional name and the agent reads it *automatically*, before you say a word, so the contract loads itself every session.

One name has become the standard: `AGENTS.md`. It's an open, vendor-neutral convention that most agent tools now read — OpenAI Codex, Cursor, the Gemini CLI, GitHub Copilot, and Claude Code. Make that your one real instruction file. A couple of tools still look for their own name, but **don't keep three full copies** — you'll edit one and forget the others, and you're right back to `FINAL_final`. Instead, make the others a one-line pointer:

```
your-workspace/
├── AGENTS.md      # the one real instruction file — every tool reads this
├── CLAUDE.md      # one line: "See AGENTS.md"
└── GEMINI.md      # one line: "See AGENTS.md"
```

Keep that file with the rest of the workspace on your shared drive — save it to Box or Dropbox, or, on GitHub, commit it — and the *whole team* inherits the same standard from one source of truth.

07 · TEACH IT YOUR VOICE

## Beyond format — teach it who you are.

Everything so far gets the AI to write *correctly*. This last move gets it to write like *you* — and toward what you actually care about. It's optional, and it's the difference between a competent draft and one nobody would guess a machine touched.

### Your voice — show, don't tell

No rule captures rhythm, word choice, and where you land your emphasis. So don't describe your voice — *demonstrate* it. Paste in two or three pieces of writing you're proud of and let the AI infer the pattern:

PASTE THIS · MATCH MY VOICE

```
Below are three things I wrote and am proud of. Don't compliment
them — study them. Describe my voice back to me: sentence length
and rhythm, the words I reach for and the ones I avoid, how I open
and how I land a point.

Then save that as a short style note in the workspace and write to
it from now on. When a draft drifts off my voice, tell me where.

[paste your writing here]
```

### A TELOS document — the *why* behind the writing

A style note teaches the AI *how* you write. A TELOS document teaches it *why*. It's one short structured file — your purpose, the missions you're on, your goals, and the values you won't trade — kept in the workspace root. With it, the AI stops writing generic-correct prose and starts writing toward what you're actually trying to do. A first pass is fifteen minutes:

PASTE THIS · DRAFT MY TELOS

```
Interview me to build a short TELOS document — my purpose, my
current missions, my goals for the next few years, and the values
I won't compromise. Ask me one question at a time, maybe eight in
all, and push back if an answer is vague.

Then draft it as a one-page file I can keep in my workspace and
point you at whenever we write something that matters.
```

### A self-authoring exercise

If that lands, go further on your own: structured writing about your past and your ideal future — the kind of guided reflection used in self-authoring programs. It does two things at once. It gives the AI a far richer picture of what drives you, and it shows *you* what you believe, in your own words, before you ever ask a machine to help you say it.

**TRAIN YOUR AI WITHOUT UN-TRAINING YOURSELF**
Handing the AI your voice and your purpose is the opposite of handing over your judgment. The clearer it is on who you are, the easier it is to catch when a draft *isn't* you — and to cut it. The goal is an assistant that sounds like your best self, not a replacement for having one.

08 · YOUR TURN

## Now you build your own.

You start from a workspace that's already built — you don't architect it, you fill it in. What you'll have at the end is a finished product and a habit that outlasts it.

| | |
|---|---|
| THE KIT | **A folder, ready to go.** The instruction file, a `sources/` folder, the reading checklist, a template for your standard. Drop a document in, walk it through, draft against the standard. |
| THE GUIDE | **This companion.** The three moves, the collaboration and voice add-ons, and every prompt to paste — keep it open beside your workspace. |
| THE PROJECT | **Your project.** Whatever complicated thing is in front of you — a program review, a policy change, a brief, a paper — grounded in sources you actually understand. The workspace you set up is how you work it, from first question to finished product. |

**And it's the first rung of a tall ladder.** The same pattern, fully built out, becomes a research project where one command reproduces an entire paper from raw data to final figures — the AI maintaining the pipeline while the researcher makes the decisions. You won't build that today, but the way up is the same three moves, just packaged for scale: your reading checklist can become a reusable *Skill* the agent loads on its own when it's relevant, and open plumbing like **MCP** (the Model Context Protocol) wires the agent straight into live sources — a citation database, your own files — so it fetches facts instead of guessing at them. And the direction is set: military staff colleges are increasingly teaching officers to build and test AI tools, not just use them, and government platforms like **GenAI.mil** are adding no-code ways to build your own assistant. The specific tools churn — the ones you'll use in a year may not exist yet — but **the discipline transfers even when the tool changes.**

### What your finished home looks like

By the end, the four-folder starter has grown into a home for everything the method produces — not just what you read and wrote, but the critiques you collected and the voice you taught it. Here's the shape to aim for:

```
your-workspace/
├── AGENTS.md      # the contract (a.k.a. INSTRUCTIONS.md)
├── sources/       # the papers & documents you read and cite
├── rubric/        # the reading checklist + your writing standard
├── drafts/        # your writing, kept in versions
├── review/        # Referee-2 critiques and your fix lists
└── voice/         # your style note + TELOS.md (voice & purpose)
```

Two of those homes are new since the starter kit: `review/`, where you keep what the skeptical reviewer found and what you changed, and `voice/`, where your style note and TELOS document live. Keep the whole thing on a shared drive with version history and it stops being a one-time trick — it's a home you (and your team) come back to for the next paper, the next brief, the next decision.

---

**Train your AI without un-training yourself.** *The goal was never to hand the thinking over. It was to stop paying the three taxes — re-explaining, re-finding, re-saying — so the thinking you do is the thinking that counts. Be a skeptic, not a cynic: set the machine up, then check everything it hands you, and never sign what you couldn't defend on your own.*

09 · RESOURCES

## Everything referenced here, in one place.

The links behind the ideas in this guide — and, at the end, the serious toolkits if you ever want to turn this into a research-grade review pipeline.

### CONTEXT ENGINEERING

Effective Context Engineering — Anthropic Applied AI team.
Anthropic Cookbook — memory, compaction, tool clearing.
Projects — persistent state, the no-code version of a home.

### READING & EVALUATING EVIDENCE

Bueno de Mesquita & Fowler, *Thinking Clearly with Data* — claim, evidence, assumptions. Start here.
Scott Cunningham — "Referee 2" with AI — adversarial empirical review.
Huntington-Klein, *The Effect* — free online; the methods bench for cause-and-effect claims.
Sextus Empiricus, *Outlines of Pyrrhonism* I.1–4 (c. 200 CE) — the Skeptics “continue to search.” Skepticism as inquiry, not cynicism.

### VOICE & WRITING

McCloskey, D. (1999). *Economical Writing* — the read-aloud test; active verbs.
Strunk & White, *The Elements of Style* — Rule 17: omit needless words.
Tongue and Quill (AFH 33-337) — active voice, BLUF, plain language.
AR 25-50 — memo format reference.

### AGENT CLIS & INSTRUCTION FILES

Claude Code, OpenAI Codex CLI, Gemini CLI — the on-your-computer agents.
`AGENTS.md` — the open, vendor-neutral instruction-file standard most tools now read.
ai-in-the-terminal — install guide for Mac / Windows / Linux.

### GOVERNMENT AI TOOLS (2026)

**GenAI.mil** — Gemini + ChatGPT, universal CAC, IL5 / CUI.
**Ask Sage** — 25+ models, retrieval, IL5.
**CamoGPT** — Army-specific, SIPRNet capability.
**M365 Copilot** — GCC-High, document-centric.

### THIS WORKSPACE & WHERE IT GOES

**The starter workspace** — the folder you fill in, provided with this guide.
**Give Your AI a Home** — this guide; keep it open beside the work.
Research-Project-Flow — the same pattern scaled to a full research project (the Rung 3 version).

### Adversarial reviewers & research tools — for the ambitious

Past a single skeptical reviewer, these are the strongest open toolkits for turning AI into a rigorous research referee. Most assume some comfort with the command line — they're a map of where this goes, not a to-do list for today.

**Cunningham — MixtapeTools / Referee 2**
The closest thing to an adversarial "Reviewer 2": separate audits for code, replication, and econometrics, plus Blindspot (what's missing) and Bibcheck (citation verification).

**Pedro H. C. Sant'Anna — claude-code-my-workflow**
A forkable academic Claude Code workflow: `/review-paper --peer`, editor/referee roles, journal calibration, and quality gates. (guide)

**Hugo Sant'Anna — Clo-Author**
A full paper-lifecycle scaffold — 18 agents, worker–critic pairs, simulated peer review, and replication compliance.

**Claes Backman — AI-research-feedback**
A compact six-agent pre-submission referee report: style, consistency, unsupported claims, math, tables, and a journal-specific adversarial referee.

**Lee Crawfurd — Claude Code research skills**
An installable bundle that blends Cunningham's Referee 2, Backman's pre-submission review, and code review into clean, reusable skills.

**OpenAIReview — Chicago Human+AI Lab**
General-purpose "paper referee as software," with worked examples of papers reviewed by multiple models.

---

© 2026 Jared Black · **Give Your AI a Home**

Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). You are free to share and adapt this guide — including for your own courses and training — with attribution.